

LAPORAN CLEANING DATA DENGAN PANDAS

1. Muat dataset `ekstrakurikuler_siswa.csv` ke Google Colab. Tampilkan `df.shape`, `df.info()`, dan `df.isna().sum()` untuk memetakan masalah pada tiap kolom.

Disini saya sudah memasukkan dataset nya ke google colab ,lalu saya melakukan pengecekan ukuran dan dilihat kolom mana saja yang bermasalah

```
[1] import pandas as pd

df = pd.read_csv('dataset_ekstrakurikuler_siswa.csv')

print("Ukuran data:", df.shape)
print()
print(df.info())
print()
print("Jumlah missing value per kolom:")
print(df.isna().sum())
```

Setelah dicek, ternyata datasetnya ada 505 baris dan 12 kolom. Pas lihat `df.info()`, kolom `nilai_rata_rata` sama `kehadiran_persen` malah kebaca sebagai teks, padahal isinya seharusnya angka. Terus dari `isna().sum()`, ketahuan ada 5 kolom yang datanya tidak lengkap

```
1] print("Jumlah missing value per kolom:")
0s print(df.isna().sum())

✓ ... Ukuran data: (505, 12)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 505 entries, 0 to 504
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ---
 0   id_pendaftaran        505 non-null   object
 1   nama_siswa            505 non-null   object
 2   jenis_kelamin         505 non-null   object
 3   kelas                 505 non-null   object
 4   ekstrakurikuler       505 non-null   object
 5   pembina               505 non-null   object
 6   tanggal_daftar       505 non-null   object
 7   no_hp_orangtua        472 non-null   float64
 8   nilai_rata_rata       479 non-null   object
 9   kehadiran_persen     487 non-null   object
10   status_pembayaran    482 non-null   object
11   catatan              228 non-null   object
dtypes: float64(1), object(11)
memory usage: 47.5+ KB
None

Jumlah missing value per kolom:
id_pendaftaran      0
nama_siswa          0
jenis_kelamin       0
kelas               0
ekstrakurikuler     0
pembina             0
tanggal_daftar      0
no_hp_orangtua      33
nilai_rata_rata     26
kehadiran_persen    18
status_pembayaran   23
catatan             277
dtype: int64
```

2. Tangani missing value: putuskan apakah `no_hp_orangtua`, `nilai_rata_rata`, `kehadiran_persen`, dan `status_pembayaran` sebaiknya di-drop atau diisi (dan diisi dengan nilai apa) — berikan alasannya.

Setelah dicek pakai `df.isna().sum()`, ada 5 kolom yang datanya kosong: `no_hp_orangtua` (33), `nilai_rata_rata` (26), `kehadiran_persen` (18), `status_pembayaran` (23), dan `catatan` (277).

Untuk `no_hp_orangtua`, `nilai_rata_rata`, `kehadiran_persen`, dan `status_pembayaran`, saya pilih **diisi (fillna)**, bukan dihapus (dropna). Soalnya kalau barisnya dihapus, data siswa itu di kolom lain (nama, ekskul, nilai, dll) ikut hilang

semua, padahal cuma satu kolom saja yang kosong. Jumlah yang kosong juga masih tergolong sedikit dibanding total 505 data, jadi sayang kalau dibuang.

Khusus kolom catatan (277 kosong, paling banyak), kami sengaja **tidak diapa-apakan** — tidak dihapus dan tidak diisi. Karena memang wajar banyak siswa tidak dikasih catatan tambahan sama panitia, itu bukan data yang error atau kurang.

```
dtype: int64

[2]
✓ Os
df['no_hp_orangtua'] = df['no_hp_orangtua'].fillna('Tidak Diketahui')

df['status_pembayaran'] = df['status_pembayaran'].fillna('Belum Lunas')

print(df.isna().sum())

id_pendaftaran      0
nama_siswa           0
jenis_kelamin        0
kelas                0
ekstrakurikuler      0
pembina              0
tanggal_daftar       0
no_hp_orangtua       0
nilai_rata_rata      26
kehadiran_persen     18
status_pembayaran    0
catatan              277
dtype: int64
```

Nilai_rata_rata sama kehadiran_persen sebenarnya juga mau diisi pakai rata-rata, tapi belum bisa langsung di nomor 2. Soalnya datanya masih berantakan, ada yang pake koma, titik, simbol %, malah ada yang isinya tulisan "belum dinilai". Jadi nanti dirapiin dulu di nomor 4 baru bisa dihitung rata-ratanya.

3. Deteksi dan hapus baris duplikat persis dengan `drop_duplicates()`. Kemudian deteksi `id_pendaftaran` yang muncul lebih dari sekali dan tentukan strategi (misalnya `keep='last'` karena dianggap data ter-update)

Disini ada dua jenis duplikat yang harus dicek terlebih dahulu, Pertama, baris yang sama persis semua kolomnya (kemungkinan si ini double-input):

```
# Cek berapa banyak baris yang sama persis (duplikat total)
jumlah_duplikat_persis = df.duplicated().sum()
print("Jumlah baris duplikat persis:", jumlah_duplikat_persis)

# Hapus baris duplikat persis
df = df.drop_duplicates()
print("Jumlah baris setelah dihapus duplikat persis:", df.shape[0])

Jumlah baris duplikat persis: 15
Jumlah baris setelah dihapus duplikat persis: 490
```

Terus saya cek lagi, ternyata ada `id_pendaftaran` yang muncul lebih dari sekali, tapi kolom lainnya beda-beda. Kemungkinan ini data yang udah di-update, misalnya kehadiran atau status bayarnya direvisi, tapi baris lamanya nggak sengaja kehapus. Nah buat kasus ini kami pake `keep='last'`, jadi yang disimpan itu baris paling akhir, soalnya dianggap itu data yang paling baru

[4]

```

jumlah_id_duplikat = df.duplicated(subset=['id_pendaftaran']).sum()
print("Jumlah baris dengan id_pendaftaran duplikat:", jumlah_id_duplikat)

contoh_id_duplikat = df[df.duplicated(subset=['id_pendaftaran'], keep=False)].sort_values('id_pendaftaran')
print(contoh_id_duplikat.head(10))

```

Re

Ple

trici

20:

...

Jumlah baris dengan id_pendaftaran duplikat: 10

id_pendaftaran	nama_siswa	jenis_kelamin	kelas	ekstrakurikuler
214	EKS20260162	Gita Anggraini	Perempuan	X RPL 1 Robotika
317	EKS20260162	Gita Anggraini	Perempuan	X RPL 1 Robotika
418	EKS20260268	Taufik Gunawan	L	XI TKJ 2 Desain Grafis
173	EKS20260268	Taufik Gunawan	L	XI TKJ 2 Desain Grafis
267	EKS20260328	Citra Kusuma	P	XII TKJ 1 Futsal
2	EKS20260328	Citra Kusuma	P	XII TKJ 1 Futsal
36	EKS20260373	Qori Hidayat	Laki-laki	XI TKJ 2 Paduan Suara
83	EKS20260373	Qori Hidayat	Laki-laki	XI TKJ 2 Paduan Suara
448	EKS20260375	Umar Wijaya	Laki-laki	X RPL 2 Voli
357	EKS20260375	Umar Wijaya	Laki-laki	X RPL 2 Voli

pembina	tanggal_daftar	no_hp_orangtua	nilai_rata_rata
214	Ibu Siti Aminah	20-07-2026	82286384066.0 95.5
317	Ibu Siti Aminah	20-07-2026	82286384066.0 95.5
418	Bpk. Farhan Hakim	2026-08-28	87815890429.0 NaN
173	Bpk. Farhan Hakim	2026-08-28	87815890429.0 NaN
267	Bpk. Farhan Hakim	2026-07-10	89519639907.0 95.6
2	Bpk. Farhan Hakim	2026-07-10	89519639907.0 95.6
36	Ibu Siti Aminah	2026-07-17	85284910256.0 95,8
83	Ibu Siti Aminah	2026-07-17	85284910256.0 95,8
448	Ibu Rina Wulandari	06-07-2026	81214870682.0 86.8
357	Ibu Rina Wulandari	06-07-2026	81214870682.0 86.8

kehadiran_persen	status_pembayaran	catatan
214	100	Belum Lunas Aktif lomba tingkat kota
317	63	Belum Lunas Aktif lomba tingkat kota
418	86	Lunas Pindah kelas semester depan
173	68%	Lunas Pindah kelas semester depan
267	63	LUNAS NaN
2	99	Belum Lunas NaN
36	63	Belum Lunas NaN
83	85	Lunas NaN
448	85	Belum Lunas NaN
357	68	Lunas NaN

[5]

```

df = df.drop_duplicates(subset=['id_pendaftaran'], keep='last')

print("Jumlah baris setelah id_pendaftaran duplikat dihapus:", df.shape[0])

```

20:

...

Jumlah baris setelah id_pendaftaran duplikat dihapus: 480

Jadi total baris yang hilang karena duplikat: 505 - 480 = 25 baris.

4 .Bersihkan kolom nilai_rata_rata (satukan format koma/titik menjadi angka desimal) dan kehadiran_persen (hilangkan simbol % lalu konversi ke angka) menggunakan kombinasi str.replace() dan pd.to_numeric().

Kolom nilai_rata_rata isinya campur: ada yang pakai koma ("74,0"), ada yang titik ("78.9"), dan ada teks "belum dinilai". Cara membersihkannya: koma diganti jadi titik dulu, baru diubah ke angka pakai pd.to_numeric() dengan errors='coerce' supaya teks yang tidak bisa diubah otomatis jadi NaN (bukan bikin program error). Setelah itu baru NaN-nya diisi dengan rata-rata.

```
# Langkah 1: ganti koma jadi titik
df['nilai_rata_rata'] = df['nilai_rata_rata'].str.replace(',', '.', regex=False)

# Langkah 2: ubah jadi angka, teks seperti "belum dinilai" otomatis jadi NaN
df['nilai_rata_rata'] = pd.to_numeric(df['nilai_rata_rata'], errors='coerce')

# Cek dulu, berapa banyak yang sekarang NaN (kosong asli + "belum dinilai" tadi)
print("Jumlah NaN di nilai_rata_rata setelah dibersihkan:", df['nilai_rata_rata'].isna().sum())

# Langkah 3: isi NaN dengan rata-rata dari data yang ada
rata_rata_nilai = df['nilai_rata_rata'].mean()
print("Rata-rata nilai (dari data yang ada):", rata_rata_nilai)

df['nilai_rata_rata'] = df['nilai_rata_rata'].fillna(rata_rata_nilai)

# Cek ulang, harusnya sekarang 0
print("Jumlah NaN setelah diisi:", df['nilai_rata_rata'].isna().sum())

...
Jumlah NaN di nilai_rata_rata setelah dibersihkan: 59
Rata-rata nilai (dari data yang ada): 83.12850356294537
Jumlah NaN setelah diisi: 0
```

Setelah diubah ke angka, jumlah NaN sempat naik jadi 59 (gabungan 26 yang memang kosong + 33 yang tadinya teks "belum dinilai"). Semua itu lalu diisi dengan rata-rata 83,13.

Kolom kehadiran_persen lebih simpel, cuma perlu hapus simbol % sebelum diubah ke angka:

```
# Langkah 1: hapus simbol % dari teksnya
df['kehadiran_persen'] = df['kehadiran_persen'].str.replace('%', '', regex=False)

# Langkah 2: ubah jadi angka
df['kehadiran_persen'] = pd.to_numeric(df['kehadiran_persen'], errors='coerce')

# Cek berapa yang NaN sekarang
print("Jumlah NaN di kehadiran_persen setelah dibersihkan:", df['kehadiran_persen'].isna().sum())

# Langkah 3: isi dengan rata-rata
rata_rata_kehadiran = df['kehadiran_persen'].mean()
print("Rata-rata kehadiran (dari data yang ada):", rata_rata_kehadiran)

df['kehadiran_persen'] = df['kehadiran_persen'].fillna(rata_rata_kehadiran)

# Cek ulang, harusnya 0
print("Jumlah NaN setelah diisi:", df['kehadiran_persen'].isna().sum())

...
Jumlah NaN di kehadiran_persen setelah dibersihkan: 17
Rata-rata kehadiran (dari data yang ada): 79.54643628509719
Jumlah NaN setelah diisi: 0
```

Setelah dibersihkan, ternyata cuma 17 baris yang NaN (bukan 18 seperti awal), karena satu baris yang tadinya kosong sudah lebih dulu terhapus di langkah duplikat.

5 .Konversi kolom tanggal_daftar yang berformat campur menjadi satu tipe datetime yang konsisten.

kolom tanggal_daftar. Pas dicek pake `.unique()`, ternyata formatnya campur-campur — ada yang 2026-08-20 (yyyy-mm-dd), ada yang 08-07-2026 (dd-mm-yyyy), ada juga yang 21/08/2026 (dd/mm/yyyy). Kalau dibiarin gini bisa bingung, misalnya 08-07-2026 itu maksudnya 8 Juli atau malah 7 Agustus.

Makanya dikonversi pake `pd.to_datetime()` dengan `format='mixed'`, biar Pandas otomatis kenalin tiap format yang beda-beda itu, sama `dayfirst=True` biar angka pertama yang ambigu dianggap tanggal duluan, soalnya gaya Indonesia emang gitu.

Hasilnya semua 480 baris berhasil dikonversi, nggak ada yang gagal (0), dan tipe datanya udah jadi `datetime64[ns]`, bukan teks lagi.

```

5 print(df['tanggal_daftar'].unique()[:20])

... ['2026-08-20' '08-07-2026' '2026-09-23' '2026-08-02' '2026-07-11'
      '21/08/2026' '2026-09-26' '2026-07-18' '2026-08-13' '06/08/2026'
      '13-08-2026' '2026-09-18' '24-07-2026' '28-07-2026' '17-08-2026'
      '04/07/2026' '11-09-2026' '19/08/2026' '12/07/2026' '01/07/2026']

# Konversi tanggal_daftar ke tipe datetime, otomatis kenali 3 format berbeda
df['tanggal_daftar'] = pd.to_datetime(df['tanggal_daftar'], format='mixed', dayfirst=True, errors='coerce')

# Cek apakah ada yang gagal dikonversi (jadi NaT = kosong versi tanggal)
print("Jumlah tanggal yang gagal dikonversi:", df['tanggal_daftar'].isna().sum())

# Cek tipe datanya sekarang, harusnya sudah datetime
print(df['tanggal_daftar'].dtype)

# Lihat contoh hasilnya
print(df['tanggal_daftar'].head(10))

Jumlah tanggal yang gagal dikonversi: 0
datetime64[ns]
0      2026-08-20
1      2026-07-08
3      2026-09-23
4      2026-08-02
5      2026-07-11
6      2026-08-21
7      2026-09-26
8      2026-07-18
9      2026-08-13
10     2026-08-06
Name: tanggal_daftar, dtype: datetime64[ns]

```

6 .Rapikan kategori pada kolom jenis_kelamin (jadikan hanya 'L'/'P') dan status_pembayaran (jadikan hanya 'Lunas'/'Belum Lunas') menggunakan df[kolom].unique() untuk memeriksa lalu str.strip()/str.lower()/mapping manual.

```

[12]
✓ Os print("Variasi jenis_kelamin:", df['jenis_kelamin'].unique())
      print()
      print("Variasi status_pembayaran:", df['status_pembayaran'].unique())

... Variasi jenis_kelamin: ['Perempuan' 'L' 'P' 'laki-laki' 'Laki-laki' 'perempuan']
      Variasi status_pembayaran: ['Lunas' 'Belum Lunas' 'lunas' 'belum lunas' 'Belum lunas' 'LUNAS']

[13]
✓ Os # --- Rapikan jenis_kelamin ---
      # Ubah dulu semua ke huruf kecil biar gampang dicocokkan
      jk_lower = df['jenis_kelamin'].str.lower()

      # Petakan ke L / P
      df['jenis_kelamin'] = jk_lower.map({
          'l': 'L',
          'laki-laki': 'L',
          'p': 'P',
          'perempuan': 'P'
      })

      print("Setelah dirapikan, jenis_kelamin:", df['jenis_kelamin'].unique())

      # --- Rapikan status_pembayaran ---
      sp_lower = df['status_pembayaran'].str.lower()

      df['status_pembayaran'] = sp_lower.map({
          'lunas': 'Lunas',
          'belum lunas': 'Belum Lunas'
      })

      print("Setelah dirapikan, status_pembayaran:", df['status_pembayaran'].unique())

... Setelah dirapikan, jenis_kelamin: ['P' 'L']
      Setelah dirapikan, status_pembayaran: ['Lunas' 'Belum Lunas']

```

Relat

Please tricks,

2026

-
-
-

2026

-
-
-

Pas dicek pake .unique(), ternyata jenis_kelamin ada 6 variasi tulisan: 'Perempuan', 'L', 'P', 'laki-laki', 'Laki-laki', 'perempuan'. Sama status_pembayaran juga ada 6 variasi: 'Lunas', 'Belum Lunas', 'lunas', 'belum lunas', 'Belum lunas', 'LUNAS'.

Cara ngerapiinnya, teksnya diubah dulu semua jadi huruf kecil pake `.str.lower()`, biar gampang dicocokkan. Habis itu baru dipetakan (mapping) pake `.map()` ke nilai yang bener.

Hasilnya, `jenis_kelamin` sekarang tinggal ['P', 'L'] doang, sama `status_pembayaran` tinggal ['Lunas', 'Belum Lunas'] doang. Udah nggak ada variasi tulisan yang aneh-aneh lagi.

7 .Simpan hasil akhir ke file baru, misalnya `dataset_ekstrakurikuler_bersih.csv`, lalu laporkan jumlah baris sebelum vs sesudah dibersihkan.

Terakhir tinggal disimpan ke file csv baru:



```
[15]: df.to_csv('dataset_ekstrakurikuler_bersih.csv', index=False)

      print("Jumlah baris sebelum dibersihkan:", 505)
      print("Jumlah baris sesudah dibersihkan:", df.shape[0])

...   Jumlah baris sebelum dibersihkan: 505
      Jumlah baris sesudah dibersihkan: 480
```

Perbandingan jumlah baris sebelum dan sesudah dibersihkan:

	Sebelum	Sesudah
Jumlah baris	505	480
Baris yang hilang	-	25 (karena duplikat, bukan karena missing value)

Berkurangnya jumlah baris murni karena penghapusan data duplikat (15 duplikat persis + 10 baris `id_pendaftaran` duplikat). Tidak ada baris yang dihapus karena missing value, karena semua data kosong kami pilih untuk diisi, bukan dihapus.